

AlertaMascota

Informe de Implementacion — Practica de Clase

Curso:	Ingenieria de Software
Fecha:	01 de Julio, 2026
Stack:	Spring Boot 3 + React 18 + MongoDB 7
Contenedor:	Docker Compose (3 servicios)
Mapas:	Leaflet.js + OpenStreetMap (sin API key)

Este informe documenta la implementacion del sistema AlertaMascota, una aplicacion web para reportar mascotas perdidas, buscarlas por criterios y gestionar una red de cuidadores. Se implementaron aproximadamente el 70% de los requisitos funcionales y no funcionales definidos, junto con 5 patrones de diseno (3 de comportamiento, 1 creacional, 1 estructural).

1. Requisitos Implementados

1.1 Modulo — Alertas de Mascotas Perdidas

ID	Descripcion	Estado
RF 1.1	Registrar mascota perdida: nombre, especie, raza, color, descripcion.	Implementado
RF 1.2	Captura de coordenadas geograficas via mapa interactivo (Leaflet).	Implementado
RF 1.3	Reporte de avistamiento anonimo con ubicacion y descripcion.	Implementado
RF 1.4	Notificacion automatica a cuidadores con alertas activas (Observer).	Implementado
RNF 1.1	Alertas procesadas y notificadas inmediatamente al guardar (<1s).	Implementado
RNF 1.2	Datos privados del dueno (telefono) no expuestos en respuestas publicas.	Implementado

1.2 Modulo — Buscador de Mascotas

ID	Descripcion	Estado
RF 2.1	Interfaz unica de busqueda con panel de filtros lateral.	Implementado
RF 2.2	Seleccion de tipo de busqueda: Todas, Por especie, Por raza.	Implementado
RF 2.3	Busqueda filtrando por especie (Perro, Gato, Ave, Conejo, Otro).	Implementado
RF 2.4	Busqueda filtrando por raza (busqueda parcial, sin distinguir mayusculas).	Implementado
RNF 2.1	Backend acepta parametros JSON estandar; motor Strategy intercambiable.	Implementado

1.3 Modulo — Red de Cuidadores

ID	Descripcion	Estado
RF 3.1	Registro bajo tres roles: Solidario, Profesional, Especializado.	Implementado
RF 3.2	Definicion de especies y tamanios de mascotas aceptadas.	Implementado
RF 3.3	Toggle para activar/desactivar recepcion de alertas del Modulo 1.	Implementado
RF 3.4	Calificacion promedio visible en perfil (campo en base de datos).	Parcial
RNF 3.1	Panel de verificacion: perfil oculto hasta que admin valide identidad.	Implementado
RNF 3.2	Servicio de cuidadores es un microservicio independiente en Docker.	Implementado

2. Evidencia de Ejecucion

2.1 Modulo de Alertas — Vista principal

Al iniciar la aplicacion, el sistema carga automaticamente datos de demo desde MongoDB (DataSeeder). Se muestran 3 mascotas perdidas: Luna (Labrador, Miraflores), Max (Siames, San Isidro) y Coco (Beagle, Barranco). Cada tarjeta muestra el estado ACTIVA, la especie, raza, color y ubicacion de referencia.

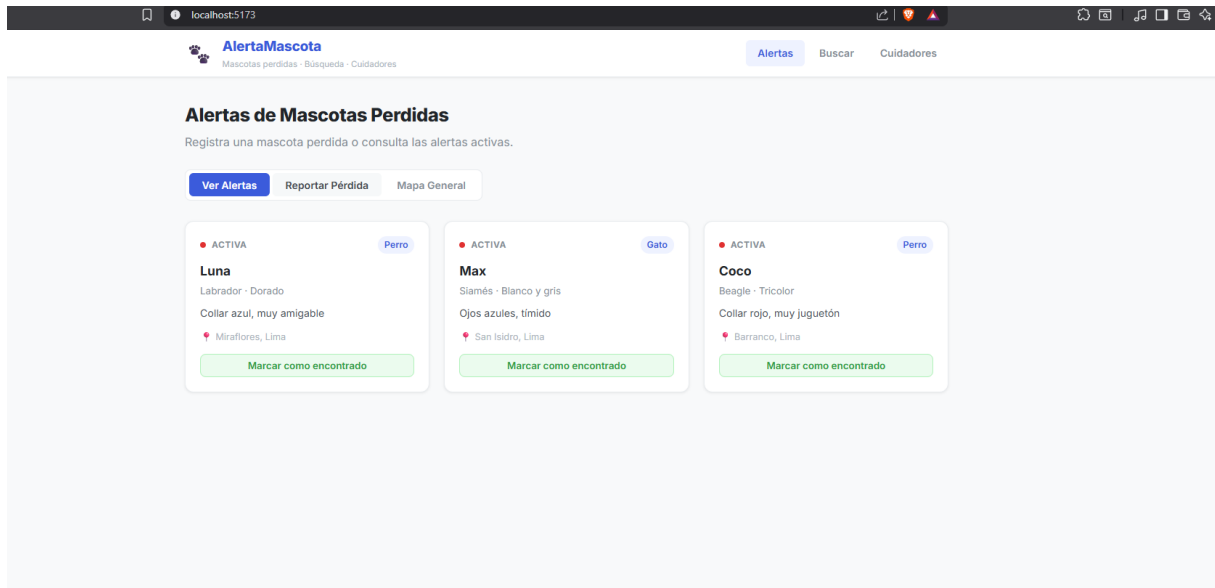


Figura 1 — Pantalla de alertas activas con datos de demo cargados desde MongoDB.

2.2 Registro de nueva alerta (RF 1.1 + RF 1.2)

Se registro una nueva mascota perdida llamada Firulais: un Labrador dorado de Ventanilla con collar negro. El usuario completa el formulario, selecciona la ubicacion en el mapa interactivo de Leaflet y envia. El sistema procesa la alerta a traves de la cadena de validacion (Chain of Responsibility) y la persiste en MongoDB. El patron Observer notifica inmediatamente a los cuidadores con alertas activas.

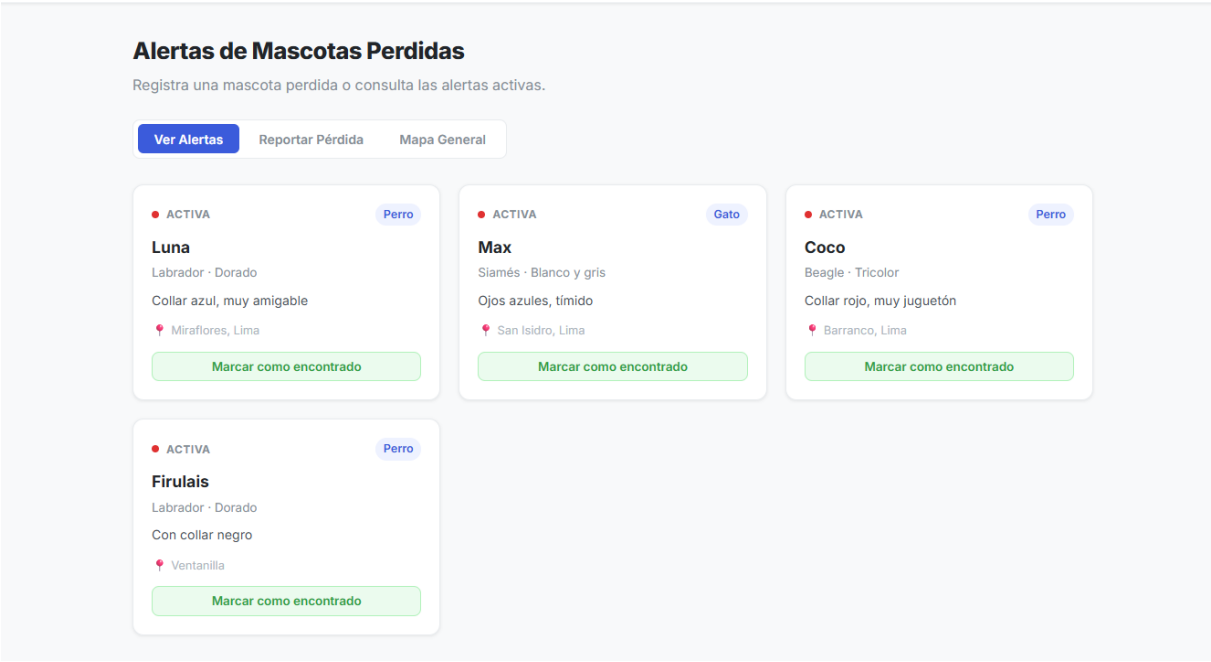


Figura 2 — Firulais aparece como cuarta alerta activa tras ser registrado.

2.3 Modulo de Busqueda — Patron Strategy en accion (RF 2.2 + RF 2.3)

El usuario selecciona el tipo de busqueda 'Por especie' y elige 'Perro'. El sistema activa la estrategia BySpeciesStrategy en el backend, que consulta MongoDB filtrando por species='Perro' y status='ACTIVE'. Se retornan unicamente Luna, Coco y Firulais (los tres perros activos), excluyendo a Max (Gato). El panel inferior izquierdo muestra explicitamente el patron Strategy aplicado.

The screenshot shows the 'AlertaMascota' web application. The header includes the logo, navigation links (Alertas, Buscar, Cuidadores), and a breadcrumb trail. The main section is titled 'Buscar Mascotas Perdidas' with a subtitle 'Filtra las alertas activas por especie o raza.' On the left, there's a 'Filtros de búsqueda' sidebar with options for 'TIPO DE BÚSQUEDA' (Todas las activas, Por especie, Por raza) and 'ESPECIE' (Perro). A 'Buscar' button is at the bottom of the sidebar. A purple box labeled 'Patrón Strategy' explains the search logic. The main content area displays three search results: 'Luna' (Dorado, Collar azul, muy amigable, Miraflores, Lima), 'Coco' (Tricolor, Collar rojo, muy juguetón, Barranco, Lima), and 'Firulais' (Dorado, Con collar negro, Ventanilla). Each result has a 'Perro' tag and a 'Labrador' tag.

Figura 3 — Busqueda filtrada por especie 'Perro'. Max (Gato) queda excluido.

2.4 Modulo de Cuidadores — Panel de Administracion (RNF 3.1)

El panel de administracion muestra todos los cuidadores registrados con su rol y estado de verificacion. Maria Lopez (Profesional) y Pedro Salas (Solidario) aparecen como Verificados. Un cuidador pendiente no aparece en el Directorio publico hasta que el administrador presione el boton Verificar, cumpliendo el RNF 3.1.

The screenshot shows the 'AlertaMascota' web application's administrator panel. The header includes the logo, navigation links (Alertas, Buscar, Cuidadores), and a breadcrumb trail. The main section is titled 'Red de Cuidadores' with a subtitle 'Cuidadores verificados disponibles para ayudar.' There are three tabs: 'Directorio', 'Registrarme', and 'Administración'. The 'Administración' tab is active. Below the tabs, there's a 'Panel de verificación' section with the subtitle 'Verifica la identidad de los cuidadores para publicar sus perfiles.' It lists two caregivers: 'María López' (Profesional, Verificado) and 'Pedro Salas' (Solidario, Verificado). Each entry has a 'Verificado' status tag.

Figura 4 — Panel de verificacion de cuidadores. Solo los verificados son visibles al publico.

3. Patrones de Diseno Implementados

3.1 Patrones de Comportamiento

COMPORTAMIENT O	Observer
Donde	service/observer/AlertObserver.java service/observer/CaregiverNotifier.java
Como funciona	Cuando se publica una alerta, AlertFacade recorre la lista de observers y llama onAlertPublished(alert) en cada uno. CaregiverNotifier consulta MongoDB y notifica a todos los cuidadores con alertsEnabled=true. En produccion enviaria email o push; en desarrollo registra en consola el nombre y rol de cada cuidador notificado.

COMPORTAMIENT O	Strategy
Donde	service/strategy/SearchStrategy.java service/strategy/SearchStrategies.java service/strategy/SearchContext.java
Como funciona	SearchContext recibe el tipo de busqueda ('all', 'species', 'breed') y delega a la estrategia correcta: AllActiveStrategy devuelve todas las activas; BySpeciesStrategy filtra por especie exacta; ByBreedStrategy hace busqueda parcial sin distinguir mayusculas. Agregar una nueva estrategia no requiere modificar el controlador (RNF 2.1).

COMPORTAMIENT O	Chain of Responsibility
Donde	patterns/AlertValidationHandler.java
Como funciona	Antes de persistir una alerta, pasa por una cadena de 3 handlers: (1) RequiredFieldsHandler valida nombre y especie obligatorios (RF 1.1); (2) LocationHandler valida que lat/lng esten presentes (RF 1.2); (3) OwnerHandler valida que el telefono de contacto no este vacio. Si cualquier handler falla, lanza excepcion y la cadena se detiene.

3.2 Patron Creacional

CREACIONAL	Builder
Donde	patterns/builder/AlertBuilder.java
Como funciona	AlertBuilder construye objetos Alert paso a paso con metodos encadenados: .petName(), .species(), .breed(), .location(), .owner(). Cada metodo valida su campo antes de asignarlo. Esto evita constructores con 10+ parametros y hace el codigo del servicio mas legible y mantenible.

3.3 Patron Estructural

ESTRUCTURAL	Facade
Donde	patterns/facade/AlertFacade.java
Como funciona	AlertFacade expone una interfaz simple al controller (createAlert, getActiveAlerts, addSighting) ocultando internamente: la construccion con Builder, la validacion con Chain of Responsibility, la notificacion con Observer y la persistencia con el repositorio MongoDB. El controller solo necesita conocer el Facade.

4. Flujo Completo — Registro de Alerta

El siguiente diagrama de texto muestra como interactuan los patrones al registrar la mascota Firulais:

```
Usuario llena formulario (nombre='Firulais', especie='Perro', lat/lng en mapa) | v
AlertController.create(body) <- Controller recibe POST /api/alertas | v
AlertFacade.createAlert(body) <- FACADE: punto de entrada unico | +--
AlertBuilder.build() <- BUILDER: construye objeto Alert | .petName('Firulais') |
.species('Perro') | .location(-11.98, -77.07, 'Ventanilla') | .owner('...',
'999...') | +-- chain.validate(alert) <- CHAIN OF RESPONSIBILITY |
RequiredFieldsHandler OK | LocationHandler OK | OwnerHandler OK | +--
alertRepository.save(alert) <- Persiste en MongoDB | +-- observers.forEach(notify)
<- OBSERVER CaregiverNotifier: -> Maria Lopez (PROFESIONAL) notificada [Pedro Salas
tiene alertsEnabled=false, no notificado]
```

Resultado en la interfaz:

Firulais aparece inmediatamente en el listado de alertas activas (Figura 2). Al buscar por especie 'Perro' en el modulo Buscar, Firulais es retornado junto a Luna y Coco (Figura 3). El backend loguea en consola que Maria Lopez fue notificada, confirmando el funcionamiento del patron Observer.